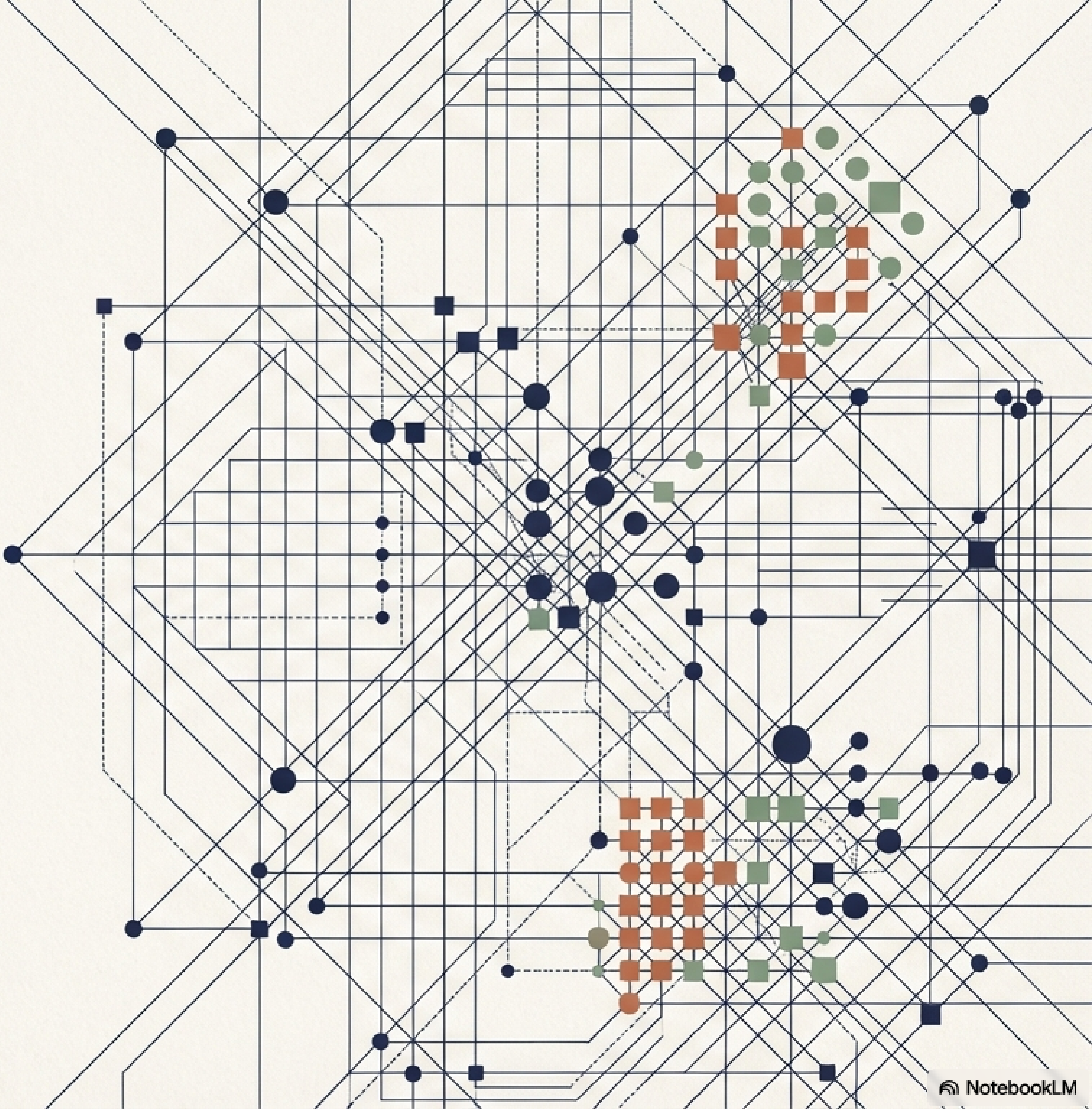
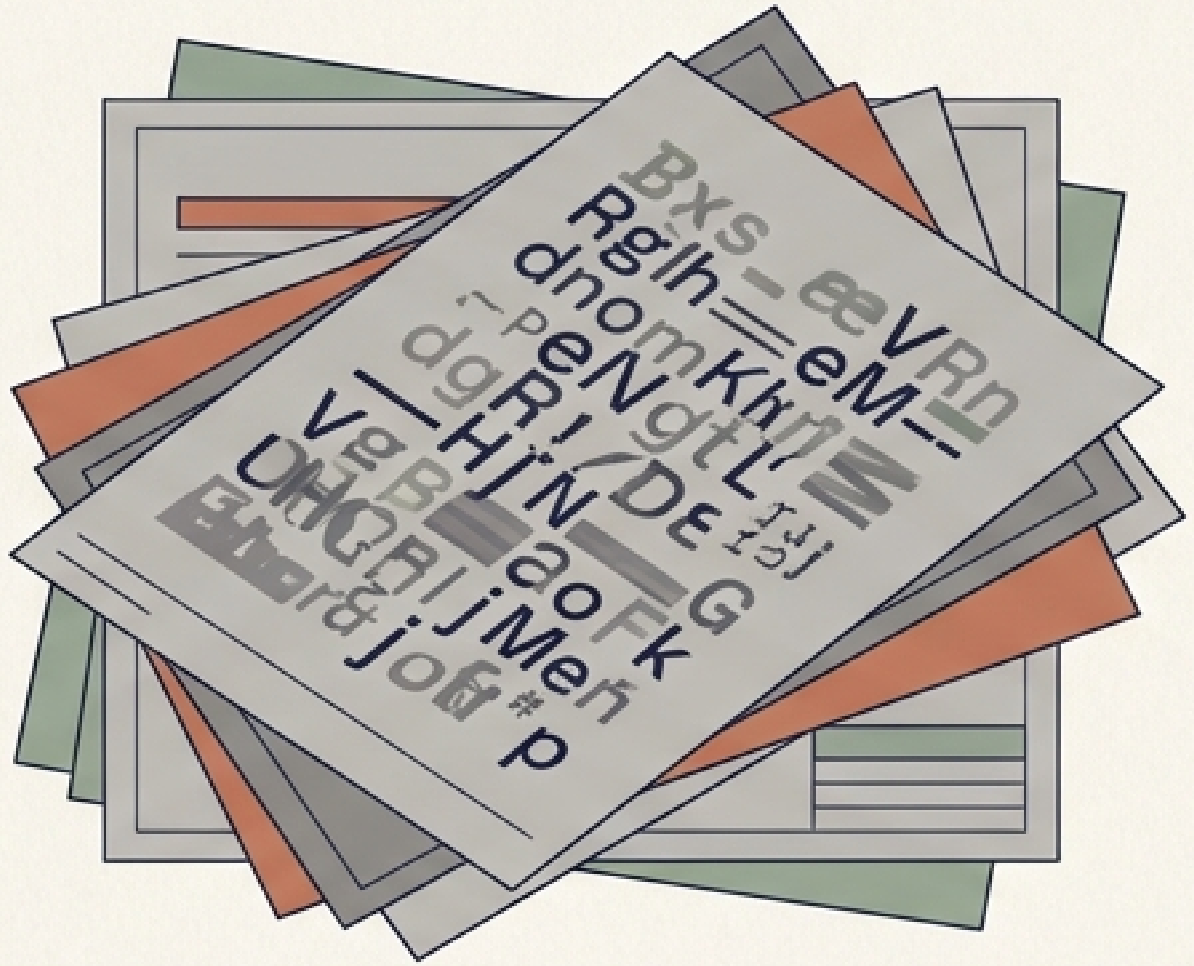
Dynamic Knowledge Graphs

Extracting structured hypergraph intelligence from unstructured PDF text.

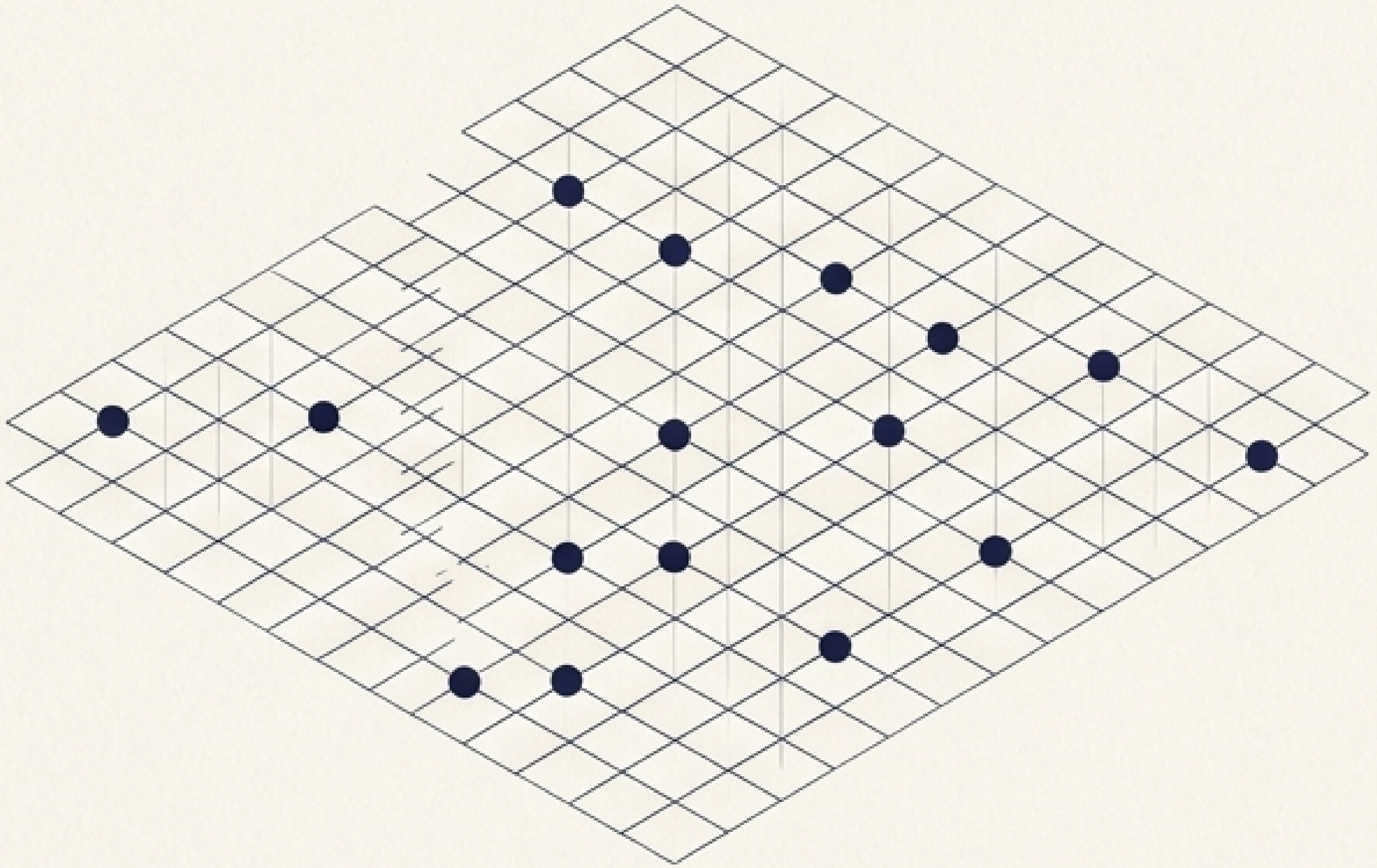


The structural limits of flat data retrieval



Unstructured Text

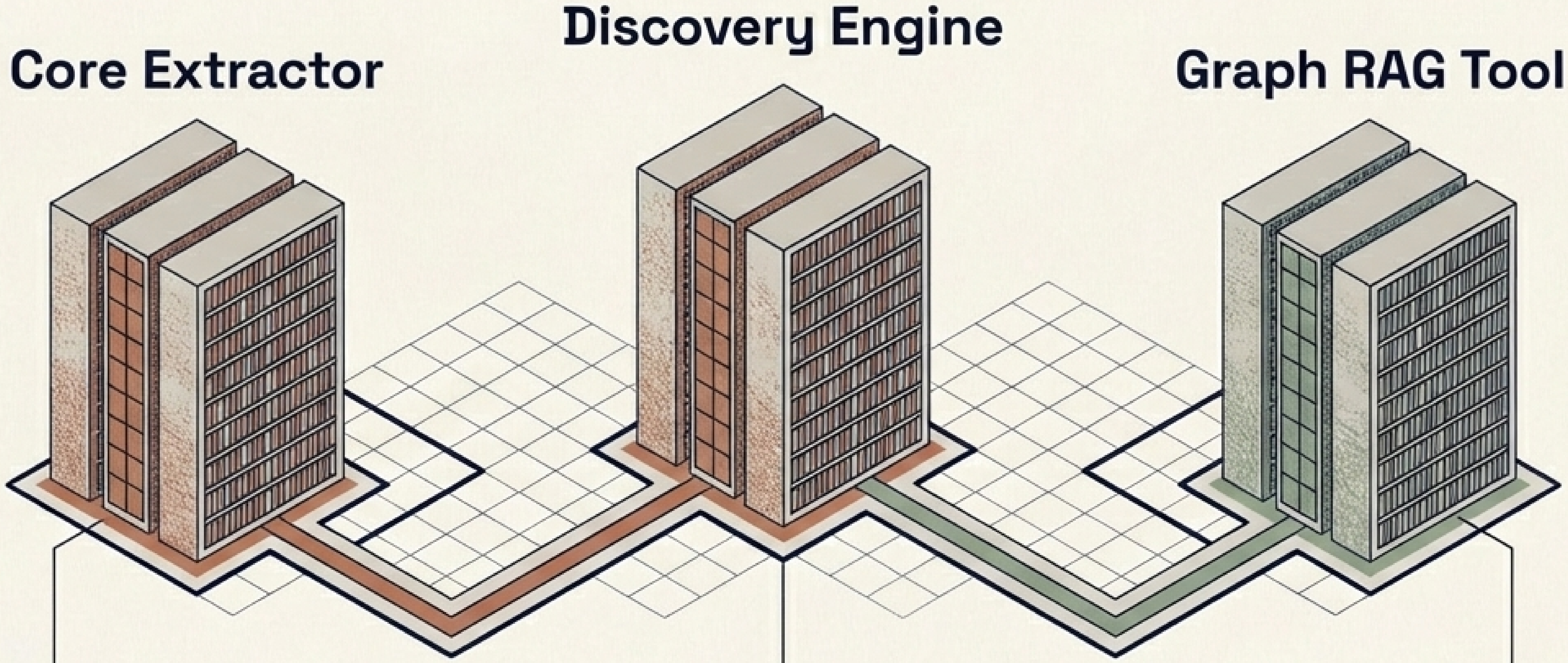
Rich in knowledge, but relationships remain trapped in rigid PDF formats.



Flat Vector Retrieval

Capable of semantic matching, but blind to multi-hop logic, hierarchies, and combinatorial insights.

An end-to-end tooling ecosystem

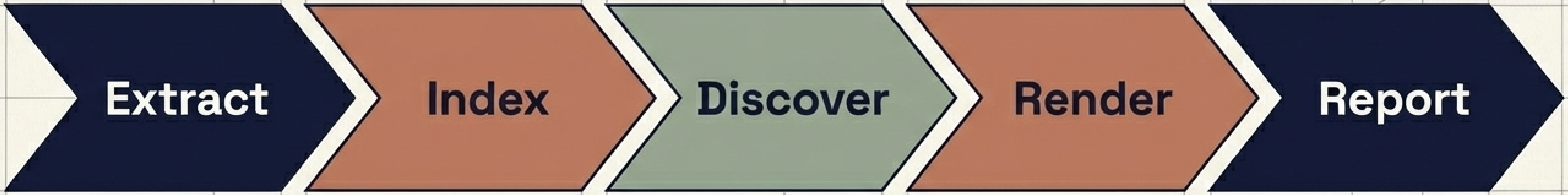


C++ and Python tooling for parsing PDFs into hypergraphs. Powered by Local LLMs, Gemini, or Azure.

55 specialized operators traversing the graph to generate categorized insights.

Interactive WebGL frontend backed by FastAPI and Kuzu DB for natural language querying.

The data transformation pipeline



Extract & Index

Parsing text and building the base graph structure.

Discover

Running operators over the graph to find hidden patterns.

Render & Report

Generating human-readable formats and runtime artifacts.

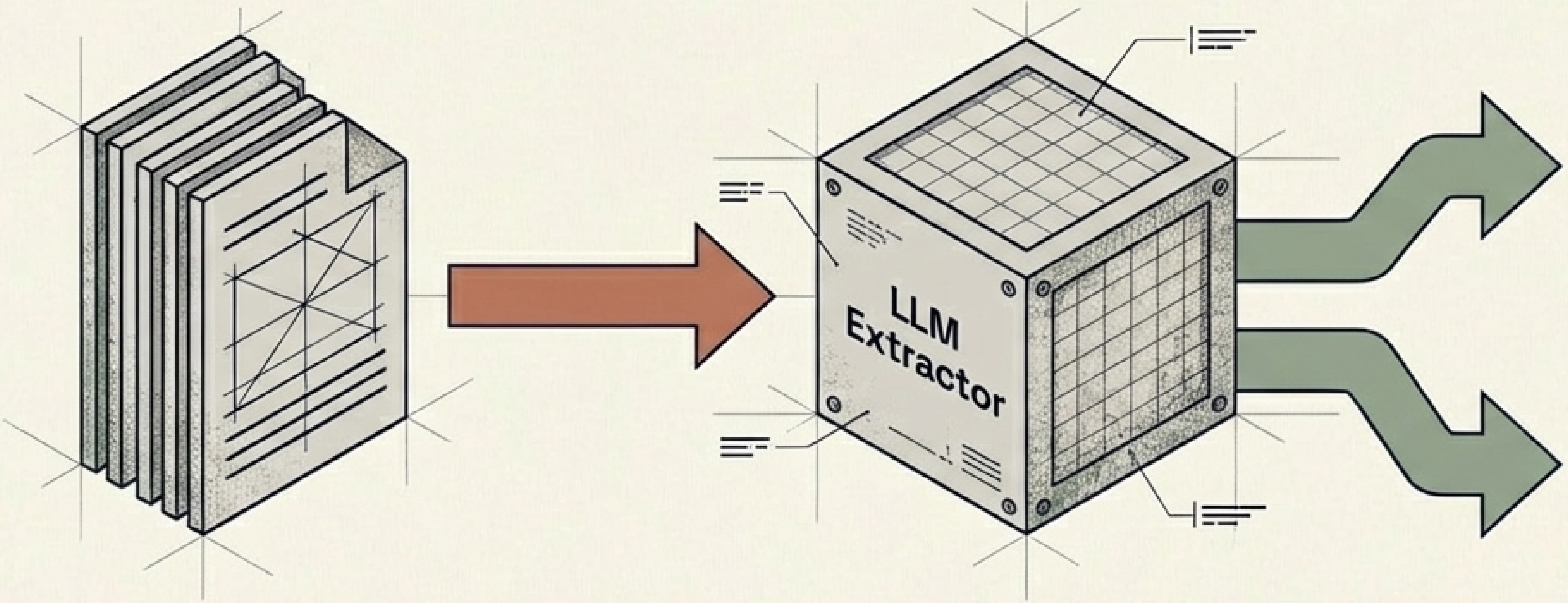
```
kg run -d <run_dir>
```

Normalizing text into structured hypergraphs

1. Ingestion

2. Extraction

3. Preprocessing & Indexing



graph.json

Key	Value
{	
"key":	"graph",
"key":	"graph.ison.com",
"key":	"value2",
...	...
}	

index.json

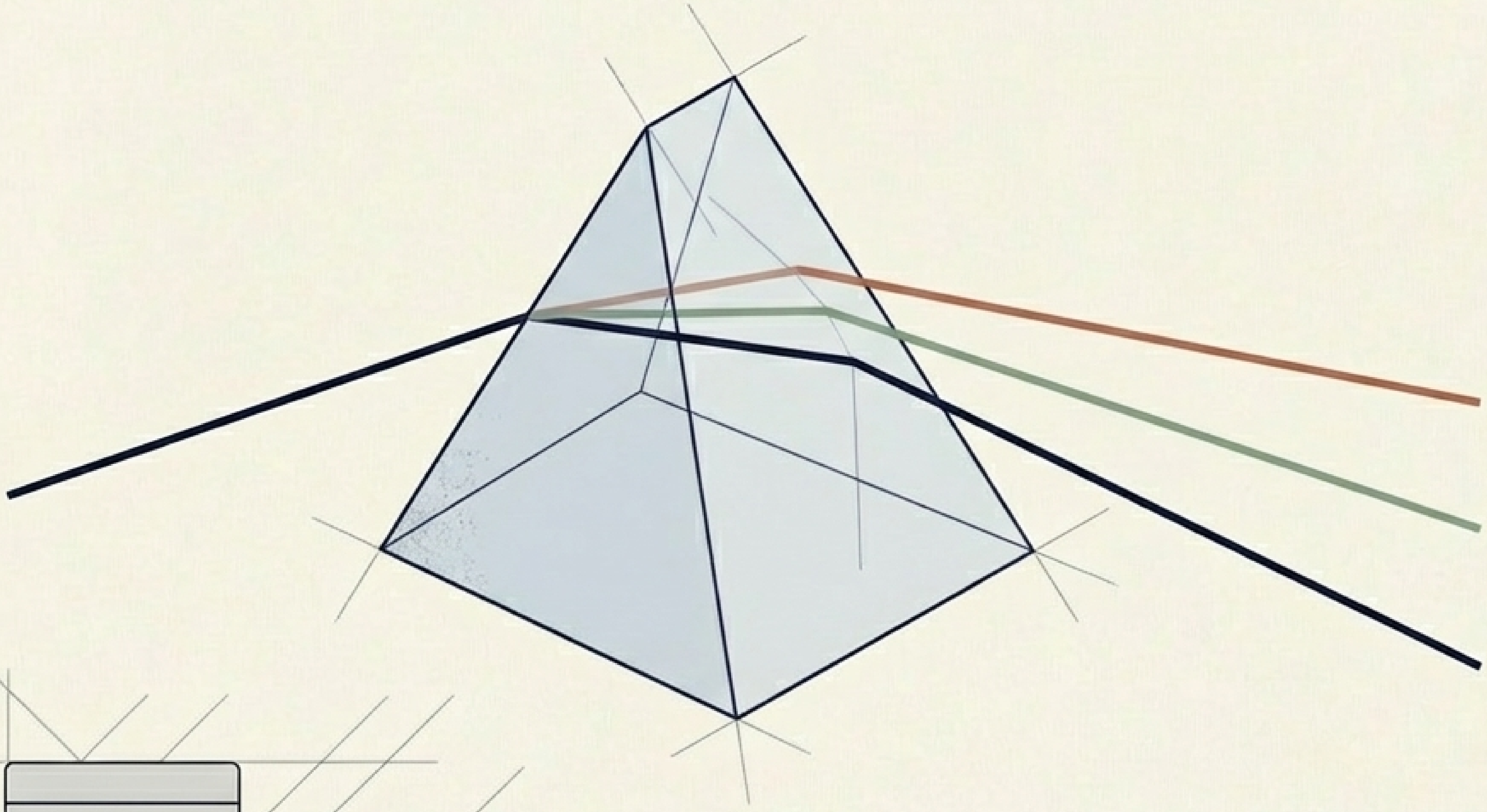
Key	Value
{	
"key":	"index",
"key":	"pzopoocies.bzg",
"key":	"value3",
"index":	"index"
...	...
}	

PDFs are processed via local or cloud LLMs (Gemini, Azure).

Entities and relationships are identified.

Using the --preprocess flag to normalize relation labels and merge entity aliases before final graph construction.

55 operators for automated discovery



Combinatorial & Exploratory

Finding hidden links and paths.

Transformational

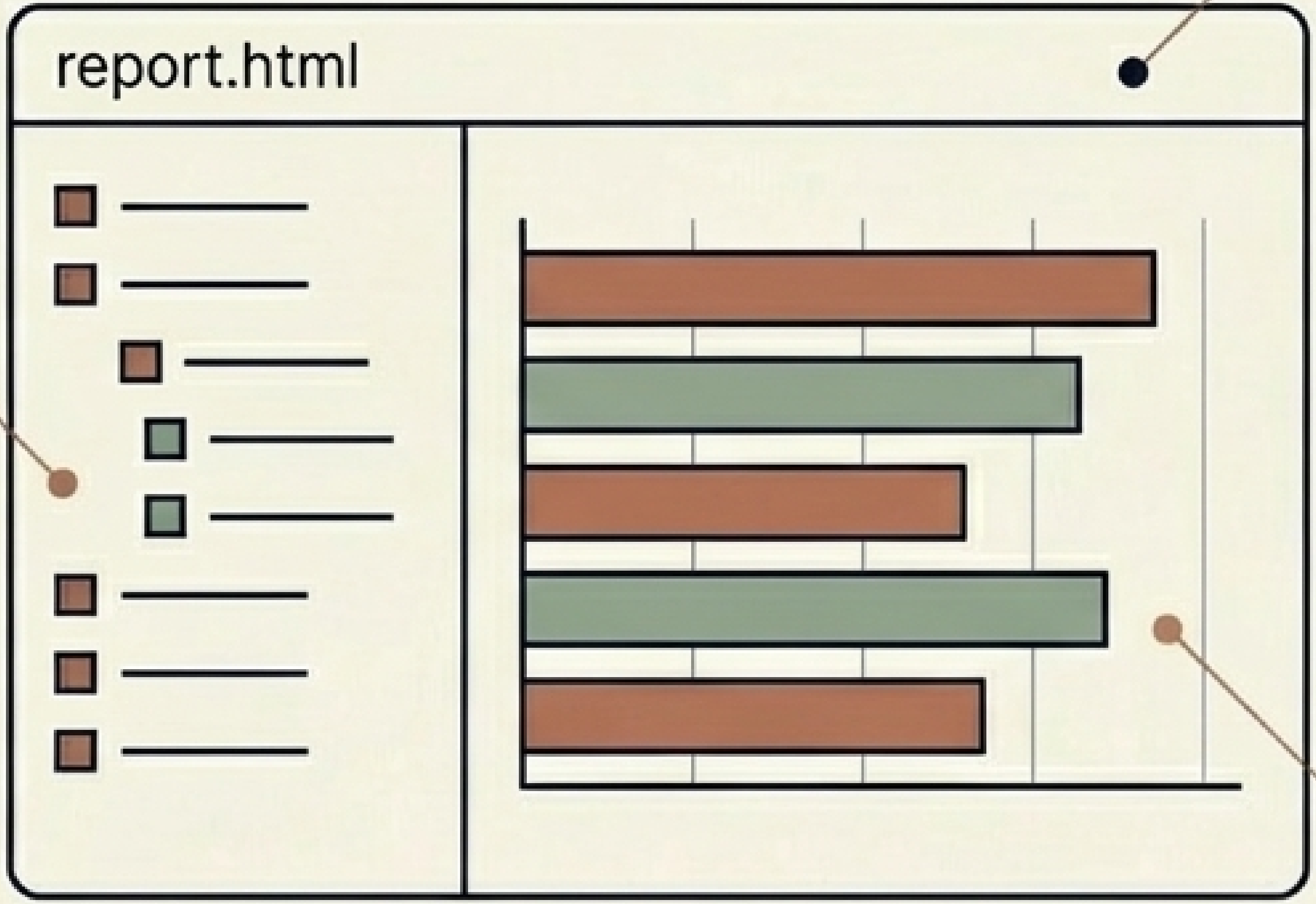
Restructuring data for new perspectives.

Advanced Analytics

Causal inference visualization, fairness-aware discovery with bias audits, and community recognition.

```
-p all
```

Intelligent runtime reporting



Modern Interface

Category-organized layout matching the core aesthetic.

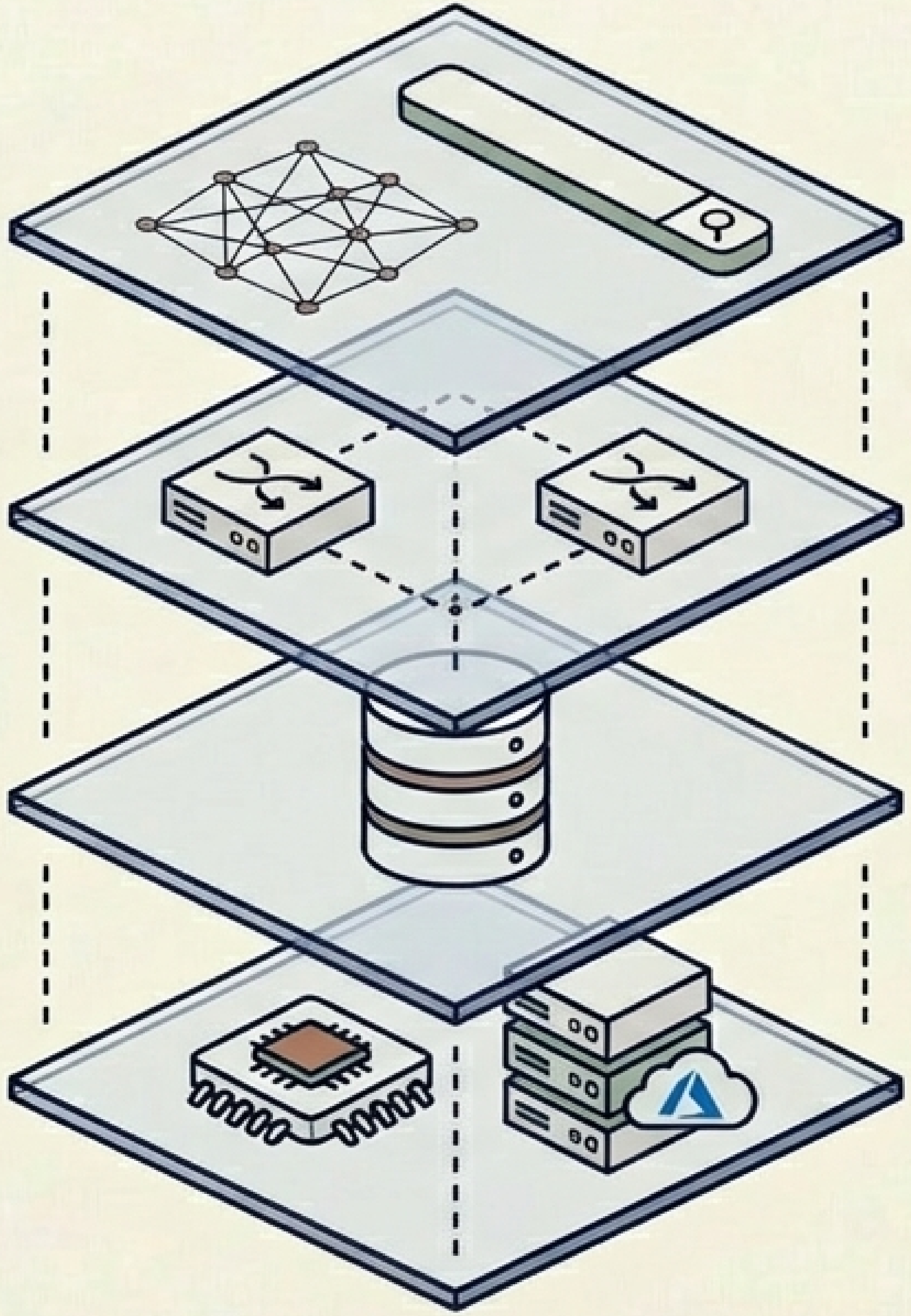
Smart Navigation

Hierarchical TOC with auto-collapsed sections.

Visual Analytics

Horizontal bar charts displaying insight distribution. Clicking chart bars jumps directly to category sections.

The Graph-RAG Tool Architecture



Frontend Layer — 3D UI & Kuzu Explorer

API Layer — FastAPI Backend enabling /api/query and /api/schema

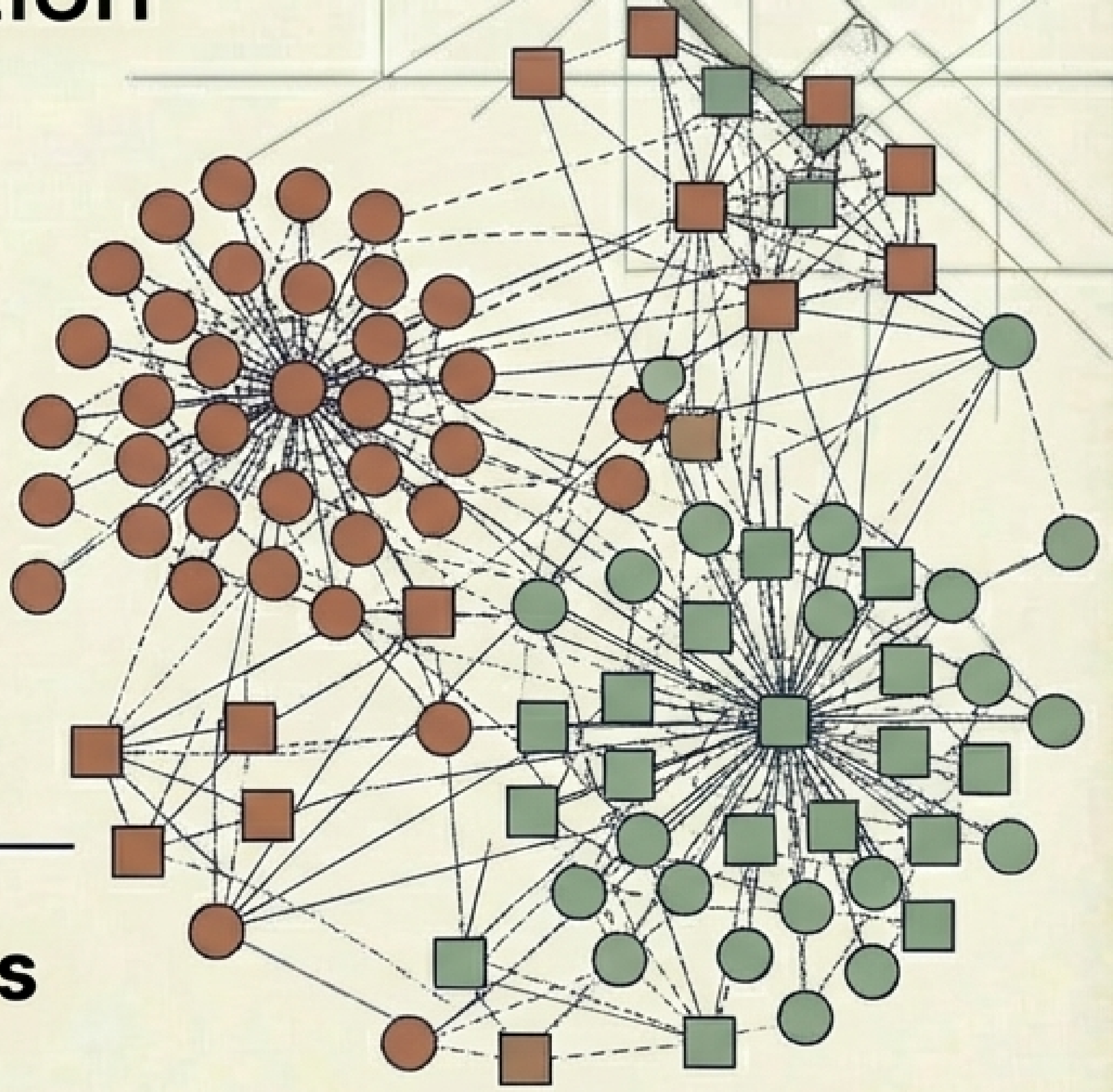
Data Layer — Kuzu DB for entity relationships

Embedding Layer — Local sentence-transformers or Cloud Azure OpenAI

Querying intelligence: The 3 RAG Modes

	Mechanism	Primary Use Case	Underlying Tech
Graph-RAG	Traverses structured entity relationships.	Best for multi-hop logic and finding connections.	Powered by Cypher queries via Kuzu DB.
Document-RAG	Retrieves exact source text context.	Best for precise citations and semantic similarity.	Powered by Vector similarity search.
Compare Mode	Side-by-side synthesis.	Best for comprehensive analysis.	Combines Cypher traversal and Vector retrieval.

Interactive visualization and exploration



WebGL Force-Directed Graph

Real-time rendering with entity clustering.

Kuzu Explorer Tooling

Direct execution of Cypher queries and targeted entity search.

Deep Pathfinding

Leveraging /api/path to find the shortest path between concepts.

Quality & Diagnostics

Accessing data quality metrics and entity hash lookups.

Extensible deployment across any environment

Platform Flexibility

- Windows (Anaconda)
- Linux/Mac (WSL/Ubuntu)

Compute Core

- C++ Core for maximum performance
- Python port for lightweight extraction

Embedding Independence

- Local LLMs (sentence-transformers) for privacy
- Gemini or Azure OpenAI for scale

From raw text to a navigable knowledge universe

```
1. Clone & Build: ./build.sh
2. Configure: .llm_config.json
3. Run Full Pipeline: kg run -p all
4. Launch App: http://localhost:8000
```

Open-source, dynamically generated, and ready for exploration.